

Date:03-08-2026
Name:MuniPardhu(192312239)

CO-03 AT -1

#

=====

== == # Experiment 4: Bayesian Belief Network

Google Colab Ready Code

#

=====

== ==

#Install pgmpy

!pip install -q

pgmpy

Import Libraries

from pgmpy.models import Discrete Bayesian

Network from pgmpy.factors.discrete import

TabularCPD

from pgmpy.inference import Variable Elimination

print("pgmpy Installed Successfully!")

#

Create Bayesian Network

#

model = Discrete

Bayesian Network([

('Rain', 'Traffic'),

('Rain', 'Late'),

('Traffic', 'Late')

])

```
# -----  
# Conditional Probability Tables  
(CPDs)  
# -----
```

```
# Rain
```

```
cpd_rain TabularCPD(  
  variable='Rain',  
  variable_card=2,  
  values=[  
    [0.7], # No Rain  
    [0.3] # Rain  
  ]  
)
```

```
# Traffic depends on Rain
```

```
cpd_traffic TabularCPD(  
  variable='Traffic',  
  variable_card=2,  
  values=[  
    [0.8, 0.2], #No Traffic  
    [0.2, 0.8] #Traffic  
  ],  
  evidence=['Rain'],  
  evidence_card=[2]  
)
```

```
# Late depends on Rain and  
Traffic
```

```
cpd_late TabularCPD(  
  variable='Late',  
  variable_card=2,  
  values=[  
    [0.9, 0.4, 0.5, 0.1], # Not  
    Late
```

```
[0.1, 0.6, 0.5, 0.9] # Late  
],  
evidence=['Rain', 'Traffic'],  
evidence_card=[2,2]  
)
```

```
#
```

```
-----  
- # Add CPDs
```

```
#
```

```
-----  
  
model.add_cpds(  
    cpd_rain,  
    cpd_traffic,  
    cpd_late  
)
```

```
#
```

```
-----  
- # Validate Model
```

```
#
```

```
-----  
  
- print("\nChecking Model...\n")
```

```
if model.check_model():  
    print("Model is VALID\n")  
else:  
    print("Model is NOT  
VALID")  
#
```

```
-----  
- # Inference
```

```
#
```

infer Variable Elimination (model)

```
print("==") print("1. Probability of being Late")
print("=====")
```

```
result1 = infer.query(variables=['Late'])
print(result1)
```

```
print("\n====") print("2. Probability of being Late")
print("Given Rain = True")
print("::====")
```

```
result2 = infer.query(
    variables=['Late'],
    evidence={'Rain':1}
)
```

```
print(result2)
```

```
print("\n====") print("3. Probability of Traffic")
print("Given Rain = False")
print("=====")
```

```
result3 = infer.query(
    variables=['Traffic'],
    evidence={'Rain':0}
)
```

```
print(result3)
print("\n=") print("4. Probability of Rain")
print("=====")
```

```
result4=infer.query(
```

```

variables=['Rain']
)

print(result4)

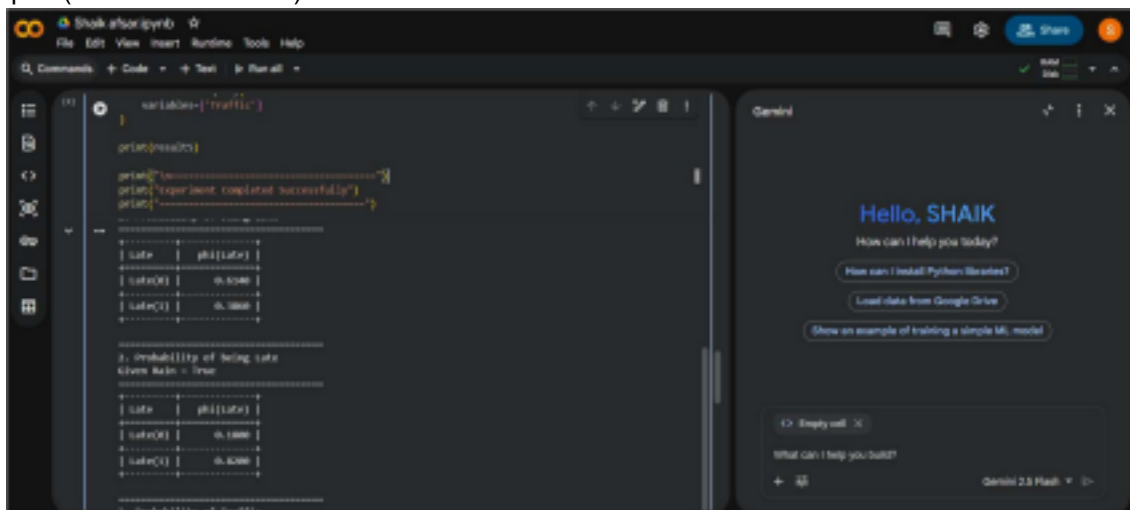
print("\n====") print("5. Probability of Traffic")
print("")

result5 = infer.query(
    variables=['Traffic']
)

print(result5)

print("\n=:=") print("Experiment
Completed Successfully")
print("=====")

```



```

#
=====

== # Experiment 6: Decision Tree Classification

# Google Colab Ready Code

#
=====

==

```

```

# Import Libraries

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt


from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import Decision Tree Classifier, plot_tree
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix


#
=====
== #Load Dataset

#
=====

== ==


iris = load_iris()
X = iris.data
y = iris.target


print("Dataset Loaded Successfully")
print("Total Samples:", len(X))
print("Features:", iris.feature_names)
print("Classes", iris.target_names)


#
=====

== == # Split Dataset

#
=====

== ==


X_train, X_test, y_train, y_test =

```

```

train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42
)

#
=====

== == # Decision Tree using Entropy

#
=====

== ==

entropy_model= Decision
Tree Classifier(
    criterion='entropy',
    random_state=42
)

entropy_model.fit(X_train, y_train)
entropy_prediction = entropy_model.predict(X_test)

print("\n=====") print("Decision Tree using
Entropy")
print("==")

print("Accuracy:", accuracy_score(y_test, entropy_prediction))

print("F1 Score", f1_score(
    y_test,
    entropy_prediction,
    average='weighted'
))

print("\nClassification Report\n")

```

```

print(classification_report(
    y_test,
    entropy_prediction,
    target_names=iris.target_names
))

```

```

print("Confusion Matrix")

```

```

print(confusion_matrix(
    y_test,
    entropy_prediction
))

```

```

#
=====
== == # Decision Tree using Gini
# =====

```

```

gini_model= Decision
TreeClassifier(
    criterion='gini',
    random_state=42
)

```

```

gini_model.fit(X_train, y_train)

```

```

gini_prediction = gini_model.predict(X_test)

```

```

print("\n===") print("Decision Tree using Gini")
print("=")

```

```

print("Accuracy:", accuracy_score(y_test, gini_prediction))

```

```

print("F1 Score", f1_score(

```



```

        y_test,
        gini_prediction,
        average='weighted'
    ))

print("\nClassification Report\n")

print(classification_report(
    y_test,
    gini_prediction,
    target_names=iris.target_names
))
print("Confusion Matrix")

print(confusion_matrix(
    y_test,
    gini_prediction
))

#
=====

# Compare Different Tree Depths
# =====

print("\n===")
print("Accuracy for Different Tree Depths")
print("")

depths = [1,2,3,4,5]

accuracy = []

for depth in depths:

    model = DecisionTreeClassifier(

```

```

        max_depth=depth,
        random_state=42
    )

    model.fit(X_train, y_train)

    prediction = model.predict(X_test)

    acc = accuracy_score(y_test, prediction)
    accuracy.append(acc)

    print("Depth=", depth,
          "Accuracy=", round(acc,4))

#
=====

# Plot Accuracy
#

=====

plt.figure(figsize=(8,5))

plt.plot(depths, accuracy, marker='o')

plt.title("Tree Depth vs Accuracy")

plt.xlabel("Tree Depth")

plt.ylabel("Accuracy")

plt.grid(True)

plt.show()

```

```

#
=====

# Plot Decision Tree

#

=====

plt.figure(figsize=(18,10))

plot_tree(
    entropy_model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)

plt.title("Decision Tree (Entropy)")

plt.show()

print("\n=====") print("Experiment Completed
Successfully")
print("==")

```



```

plt.figure(figsize=(10, 10))
plt.show()

print("=====")
print("Experiment completed successfully")
print("=====")

decision tree using entropy
Accuracy : 0.9333333333333333
F1 Score : 0.9333333333333333

classification report

```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.93	1.00	0.96	13
virginica	1.00	0.93	0.96	13
accuracy			0.96	45
macro avg	0.96	0.97	0.97	45
weighted avg	0.96	0.96	0.96	45

```

confusion matrix
[[ 0  0  0]
 [ 0 13  0]
 [ 0  1 12]]

```

